

캡스톤디자인 2차 보고서

Team 27 - Matcha Tonic

GitHub: <https://github.com/2025-MatchaTonic>

1. Team Info (팀 정보)

1.1 과제명

ProMate: 조율자 부재로 의사결정과 협업 관리에 어려움을 겪는 초보 대학생 팀 프로젝트를 위한 AI 기반 단계별 의사결정 및 역할별 업무 가이드 지원 서비스

1.2 팀 정보

- 팀 번호/이름: 27-Matcha Tonic
- 트랙: 산학
- 지도교수: 오유란 교수님

1.3 팀 구성원

구성원	담당 역할	세부 책임
-----	-------	-------

지연경 (2371063)	팀장, AI	프로젝트 총괄, 일정 관리, AI 기능 구현 및 검증
김서연 (2371013)	백엔드	API 설계 및 구현, 서버/클라우드 관리
조수아 (2371060)	프론트엔드, 디자인	UI/UX 설계, 프론트엔드 구현, 사용자 화면 구성

2. Project-Summary (과제 요약)

2.1 문제 정의

대학생 소프트웨어 팀 프로젝트는 단순히 기획이나 개발 수행 자체보다, 프로젝트 경험 부족과 역할 정의의 불명확성으로 인해 조율자가 부재한 상태에서 협업이 이루어지는 경우가 많습니다. 특히 소프트웨어 프로젝트의 경우 기획-설계-개발-테스트로 이어지는 단계적 흐름 속에서 의사결정 조정, 역할 분배, 일정 관리, 문서화와 같은 프로젝트 운영 활동이 필수적으로 요구됩니다.

그러나 이를 체계적으로 관리할 조율자가 없는 경우, 팀은 다음과 같은 어려움을 겪습니다.

- 프로젝트 초기 착수 단계에서 무엇을 먼저 결정해야 하는지 기준이 없어 합의 과정에 많은 시간 소요
- 회의 과정에서 도출된 결정사항이 명확히 정리되지 않아 누락·중복·혼선 반복
- 기능 정의, 역할 분담, 일정 계획 등 선행 작업이 정리되지 않아 이후 개발 과정에서 재작업 증가
- Notion, GitHub 등의 협업 도구를 사용하더라도 운영 기준이 없어 비효율 발생

즉, 문제의 핵심은 단순한 협업 도구의 부재가 아니라, 프로젝트를 어떻게 진행해야 하는지에 대한 업무 가이드와 운영 기준의 부재에 있습니다.

따라서 본 과제는 조율자가 없는 대학생 소프트웨어 팀 프로젝트 환경에서, AI가 프로젝트 초기 의사결정

구조를 제시하고 핵심 결정사항을 정리하며 협업 구조와 운영 기준을 설계해주는 서비스의 필요성에서 출발하였습니다.

세부 문제 정의

- **초기 착수 지연:** 팀 주제 선정, 역할 정리, 일정 합의가 늦어짐
- **의사결정 혼선:** 논의 결과가 문서로 정리되지 않아 기준이 흔들림
- **협업 관리 부재:** 일정, 역할, 산출물 관리가 체계적으로 이루어지지 않음

2.2 기존연구와의 비교

기존의 협업 도구(Notion, GitHub, Discord 등)는 문서 작성, 형상 관리, 커뮤니케이션 및 정보 공유를 지원하지만, 프로젝트 초기 단계에서 무엇을 먼저 정해야 하는지, 논의를 어떤 순서로 구조화해야 하는지, 초보 팀원이 역할별로 무엇을 해야 하는지까지 직접 안내하지는 못합니다.

Asana는 프로젝트와 업무를 계획하고 추적하며 일정, 목표, 워크플로우를 관리하는 것이 주된 기능입니다. 그러나 사용자가 프로젝트 주제, 목표, 역할과 정하지 못한 초기 상황에서, 워크플로우 정리를 하기에는 매우 어렵습니다.

Confluence는 팀 문서 작성, 협업 페이지 구성, 템플릿 기반 문서화를 지원하는 협업 워크스페이스입니다. 하지만 프로젝트 초기에 어떤 프로세스로 프로젝트를 진행해야하는지 단계별로 안내하는 데에는 한계가 있습니다. 또한 문서 구조를 직접 설계하고 관리해야 하므로, 경험이 적은 소규모 팀에게는 오히려 운영 부담과 초기 오버헤드가 발생할 수 있습니다.

Jira는 이슈 관리, 애자일 보드, 프로젝트 추적, 계획·진행·보고 기능을 제공하는 도구로서, 주로 개발 및 업무 수행 과정에서의 체계적 관리를 할 수 있도록 합니다. 그러나 Jira를 효과적으로 사용하기 위해서는 일정 수준의 문서 관리 및 프로젝트 경험이 필요합니다. 따라서 경험이 부족한 초보 팀에게는 초기 설정과 운영 자체가 부담이 될 수 있습니다.

또한, 일반적인 생성형 AI는 질의응답과 아이디어 제안은 가능하나, 프로젝트 운영에 최적화되어 있다고 보기에는 어렵습니다. 특히 범용적인 목적으로 사용하기 때문에, 현재 프로젝트와 직접 관련이 없는 정보나 사용자 메모리에 저장된 다른 맥락이 함께 반영될 가능성이 있습니다.

반면, ProMate는 단순한 협업 도구나 범용 AI 챗봇이 아니라, 소프트웨어공학 지식과 프로젝트 운영 관점을 바탕으로 프로젝트 초기 단계에서 필요한 의사결정을 구조화하고, 목표·역할·일정·산출물 등의 필수 항목을 단계적으로 정리할 수 있도록 지원하는 AI 서비스입니다. 이를 통해 경험이 부족한 팀도 프로젝트를 보다 체계적이고 안정적으로 시작하고 운영할 수 있습니다. 또한 채팅을 통해 회의와 요약이 동시에 가능하므로 회의록 도구, 기록 도구 등 여러 도구를 개별적으로 사용해야 하는 불편을 줄일 수 있습니다.

2.3 제안 내용

ProMate는 조율자 없이 프로젝트를 진행해야 하는 초보 대학생 팀을 위해, 단계별 의사결정과 역할별 업무 가이드를 제공하여 협업 혼선과 운영 부담을 줄여주는 AI 기반 프로젝트 설계 지원 서비스입니다.

ProMate는 프로젝트 초기 운영에서 자주 발생하는 의사결정 누락과 협업 준비의 비효율을 해결하기 위해, AI 챗봇 기반의 단계별 가이드 방식을 제안합니다.

1. 프로젝트 시작 시 챗봇이 사용자에게 프로젝트 주제 존재 여부를 먼저 확인합니다.
2. 응답 결과에 따라 질문 흐름을 분기하여 목표, 팀 인원, 역할, 마감일, 산출물 등 프로젝트 초기에 필요한 필수 항목을 순차적으로 수집합니다. 또한 미정 항목을 점검하고, 정해지지 않은 요소를 중심으로 다음 의사결정을 유도합니다.
3. 이를 통해 조율자가 없는 초보 팀도 대화 흐름 속에서 프로젝트에 필요한 정보를 빠짐없이 정리할 수 있도록 지원합니다.
4. 필수 항목이 일정 수준 이상 채워지면, Notion 워크스페이스와 프로젝트 운영 템플릿을 자동 생성하여 협업 환경을 빠르게 구성할 수 있도록 함으로써 초기 세팅 부담을 줄입니다.

즉, ProMate는 단순히 결과물을 생성하는 서비스가 아니라, 프로젝트를 어떻게 시작하고, 어떻게 정리하며, 어떻게 협업 구조를 세울 것인지를 안내하는 서비스로 설계되었습니다.

2.4 기대 효과 및 의의

1) 프로젝트 착수 속도 향상

AI 챗봇이 프로젝트 시작 시 필요한 핵심 항목을 순차적으로 수집하고, 무엇을 먼저 정해야 하는지 안내함으로써 팀이 초기 막막함 없이 빠르게 방향을 잡고 실행에 돌입할 수 있습니다.

2) 의사결정 과정의 명확화

회의 및 채팅에서 도출된 결정사항을 구조화하여 기록함으로써, 역할·일정·담당자에 대한 기준을 명확히 공유할 수 있고 재논의와 혼선을 줄일 수 있습니다.

3) 협업 도구 활용 효율 증가

Notion 워크스페이스와 프로젝트 템플릿을 자동 생성함으로써 협업 도구 세팅에 소요되는 시간을 줄이고, 초보 팀도 일관된 기준 아래에서 협업을 시작할 수 있습니다.

4) 역할 이해도 및 협업 역량 향상

각 역할에서 무엇을 해야 하는지에 대한 가이드를 제공함으로써, 팀원들이 맡은 역할을 더 빠르게 이해하고 이후 프로젝트에서도 활용 가능한 협업 역량을 기를 수 있습니다.

5) 과제의 의의

본 과제는 단순한 협업 보조 도구 개발이 아니라, 조율자 부재라는 대학생 팀 프로젝트의 구조적 문제를 AI 기반으로 해결하려는 시도라는 점에서 의의가 있습니다. 특히 초보 팀이 스스로 협업 구조를 형성하도록 지원한다는 점에서 실용성과 교육적 가치를 동시에 가집니다.

2.5 주요 기능 리스트

기능	기능 상세 설명	예시
AI 챗봇 기반 대화형 프로젝트 정보 수집 기능	프로젝트 시작 시 챗봇이 사용자에게 프로젝트 주제 존재 여부를 먼저 확인하고, 응답 결과에 따라 후속 질문 흐름을 분기하는 기능입니다. 주제가 없는 경우에는 주제 설정을 위한 예시와 추천안을 제공하고, 주제가 있는 경우에는 해당 주제를 바탕으로 목표, 팀 인원, 역할, 마감일, 산출물 등 프로젝트 착수에 필요한 정보를 순차적으로 수집합니다. 이를 통해 사용자는 정해진 입력 폼을 한 번에 작성하지 않고도, 질문에 답하는 방식으로 필요한 정보를 입력할 수 있습니다.	팀이 “AI 기반 앱을 만들자”는 수준의 아이디어만 가지고 있는 경우, AI 챗봇은 “프로젝트의 구체적인 목표는 무엇인가요?”, “주요 기능은 어떤 것으로 설정하시겠습니까?”, “팀원별 역할을 어떻게 나눌까요?”, “마감 일정은 언제로 설정할까요?”와 같은 질문을 순차적으로 제시합니다. 이를 통해 막연한 논의를 프로젝트 착수에 필요한 정보 수집 과정으로 전환할 수 있습니다.

프로젝트 정보 정리 및 상태 관리 기능	챗봇과의 대화 과정에서 입력된 내용을 제목, 목표, 팀 인원, 역할, 마감일, 산출물과 같은 항목 체계에 맞추어 세션 요약에 반영하는 기능입니다. 사용자의 답변이 반영될 때마다 해당 항목이 갱신되며, 아직 입력되지 않은 항목은 미정 상태로 유지됩니다. 또한 시스템은 필수 항목의 입력 여부를 확인하여 다음에 물어볼 질문을 결정하고, 템플릿 생성 단계로 넘어갈 수 있는 최소 조건이 충족되었는지를 판단합니다.	팀원들이 대화 과정에서 “프론트엔드는 A가 맡을게”, “백엔드는 B가 맡고 API는 C가 담당하자”, “다음 주 금요일까지 1차 기능 구현하자”라고 입력하면, 시스템은 이를 바탕으로 역할, 담당자, 일정 정보를 세션 요약에 반영합니다. 이후 아직 입력되지 않은 목표나 산출물 항목이 있다면 해당 내용을 다음 질문으로 안내하여 프로젝트 정보를 점진적으로 완성합니다.
Notion 워크스페이스 및 템플릿 자동 생성 기능	세션 요약에 포함된 필수 항목이 최소 기준 이상으로 채워지면, 시스템은 이를 바탕으로 Notion에서 바로 사용할 수 있는 워크스페이스와 기본 템플릿을 자동 생성하는 기능입니다. 생성 결과에는 프로젝트 개요, 회의 기록, 역할 분담, 일정 관리, 산출물 정리 등에 활용할 수 있는 기본 페이지 구조가 포함되며, 사용자는 프로젝트 시작 시 협업 환경을 빠르게 세팅할 수 있습니다.	프로젝트 초기 설정이 완료되면, 시스템은 프로젝트 개요 페이지, 팀원 및 역할 정리 페이지, 회의록 템플릿, 일정 관리 보드, 할 일 목록(Task 관리) 등으로 구성된 Notion 워크스페이스를 자동 생성합니다. 이때 각 문서는 단순한 빈 양식이 아니라 앞서 수집된 프로젝트 정보와 의사결정 내용이 반영된 상태로 제공되어, 팀이 별도의 세팅 없이 바로 협업을 시작할 수 있습니다.

3. Project-Design (과제 설계)

3.1 요구사항 정의

1) 기능 요구사항

본 시스템은 다음 요구사항을 만족해야 합니다.

- 사용자가 프로젝트 주제 존재 여부에 따라 다른 질문 흐름으로 진입할 수 있어야 함.
- 프로젝트 목표, 팀 인원, 역할, 마감일, 산출물 등 핵심 정보를 대화형으로 수집할 수 있어야 함.
- 입력된 내용을 세션 요약 형태로 구조화하고, 누락된 항목을 기준으로 다음 질문을 제시할 수 있어야 함.
- 회의 및 채팅에서 도출된 핵심 결정사항을 정리하여 문서에 반영 가능한 형태로 변환할 수 있어야 함.
- 최소 필수 항목이 충족되면 Notion 워크스페이스 및 기본 템플릿을 자동 생성할 수 있어야 함.
- 사용자 역할별 업무 가이드와 프로젝트 진행에 필요한 구조를 제공할 수 있어야 함.

2) 비기능 요구사항

- 프론트엔드, 백엔드, AI 서버 간 연동이 안정적으로 이루어져야 함.
- 사용자가 별도의 설명 없이도 흐름을 이해할 수 있도록 UI/UX가 직관적이어야 함.
- 프로젝트 초기 데모 및 발표 상황에서도 주요 기능이 안정적으로 재현되어야 함.
- 문서 및 템플릿 생성 결과물이 실제 협업에 활용 가능한 수준이어야 함.
- AI 응답 속도와 workflow 흐름은 지속적으로 개선 가능해야 함.

3) 타겟 사용자 요구 반영

본 서비스의 주요 타겟은 다음과 같습니다.

- 팀 내 의견을 정리하고 논의를 구조화할 사람이 부족한 초보 대학생 팀
- 프로젝트 초기 주제와 목표를 구체화하지 못해 진행이 지연되는 팀
- 팀 프로젝트 경험이 부족해 회의, 역할, 일정 관리가 체계적이지 않은 팀
- Notion 등 협업 도구는 사용하지만 공통된 워크스페이스 틀이 없는 초보 대학생 팀

4) 고객 페르소나

구분	내용
----	----

이름	김하은(21세,여)
배경	컴퓨터공학과 3학년. 교내 해커톤에 5인 팀으로 참가해 팀장을 맡고 있습니다. 아이디어를 내고 실행으로 옮기는 속도는 빠르지만, 프로젝트를 시작할 때마다 회의 진행, 의견 정리, 역할 분담, 일정 확인, 협업 도구 세팅까지 함께 맡게 되어 부담이 큰 상태입니다.
당면 과제	팀 내에서 다양한 의견이 제시되지만 이를 정리해 결론으로 묶는 과정이 늦어 회의 시간이 길어집니다. 또한 협업 도구를 사용하고 있음에도 공통된 기록 양식과 운영 기준이 없어 프로젝트 시작 단계에서 불필요한 시간 소모가 발생하고, 팀원들이 역할별 해야 할 일을 명확히 파악하지 못해 초반 진행이 지연됩니다.
사용자 요구	프로젝트 초기에 주제, 목표, 담당자, 마감일과 같은 핵심 사항을 빠르게 정리하고 싶습니다. 또한 Notion을 기반으로 회의록, 역할 분담, 일정 기록에 사용할 수 있는 공통 워크스페이스 템플릿을 바로 생성해 팀원들과 동일한 기준으로 협업을 시작하고 싶습니다. 아울러 팀원들이 각자 맡은 역할에서 무엇을 해야 하는지 쉽게 파악할 수 있는 기준도 필요합니다.
주요 니즈	회의를 오래 끌지 않고 핵심 사항을 빠르게 정리하고 싶습니다. 프로젝트 시작 시 협업 환경을 바로 세팅하고 싶습니다. 팀원들이 각자 해야 할 일을 빠르게 이해해 초반에 방향하는 시간을 줄이고 싶습니다.

3.2 전체시스템 구성

1) 시스템 구성 개요

본 시스템은 프론트엔드, 백엔드, AI 서버, 외부 연동 모듈로 구성되며, 각 구성 요소는 다음과 같은 역할을 수행합니다.

프론트엔드

- 사용자 입력 인터페이스 제공
- 팀 채팅 및 프로젝트 진행 현황 화면 제공

- 사용자와 시스템 간 상호작용 지원

백엔드

- 사용자 인증, 프로젝트/세션 관리
- 채팅 데이터 처리 및 저장
- 프론트엔드와 AI 서버 간 요청 및 흐름 중계
- AWS 기반 서버 배포 및 Docker 컨테이너 운영
- GitHub Actions를 통한 CI/CD 파이프라인 구성

AI 서버

- 사용자 대화 분석을 통한 프로젝트 상태 파악
- 의사결정 지원 및 대화 응답 생성
- 템플릿 내부 예시 내용 생성

외부 연동

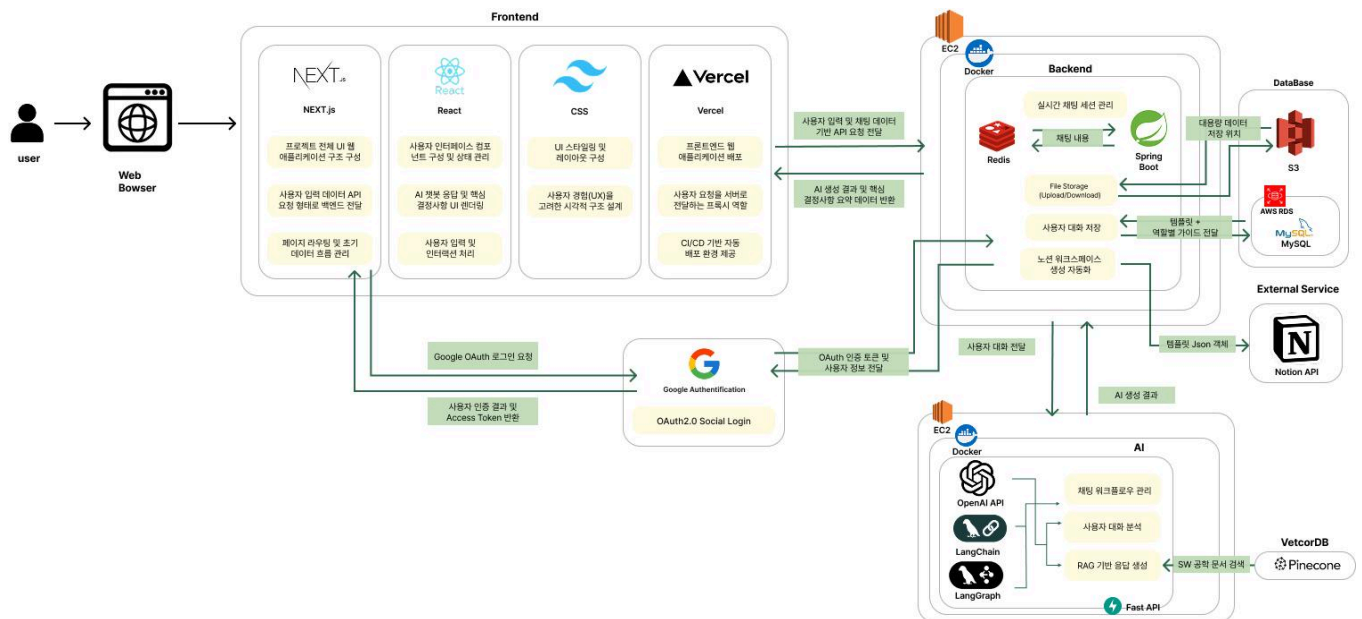
- Notion API 기반 워크스페이스 및 템플릿 생성

2) 개발 환경 및 기술 스택

구분	적용 기술	설명
Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS, Zustand, STOMP, SockJS, Vercel	사용자 인터페이스 구현, 상태 관리, 실시간 채팅 기능 및 프론트엔드 배포에 활용
Backend	Java 21, Spring Boot 3.4, Spring Security, OAuth2 Client, JWT, Spring Data JPA, Spring WebSocket , STOMP	인증, 프로젝트·세션 관리, 채팅 데이터 처리, 실시간 메시징 기능 구현에 활용

AI	FastAPI, Uvicorn, LangChain, OpenAI API, LangGraph, Pinecone	사용자 대화 분석, LLM 기반 응답 생성, 템플릿 생성, RAG 기반 검색 기능 구현에 활용
Database / Storage	MySQL, Amazon RDS, Amazon S3	관계형 데이터 저장과 정적 파일 및 리소스 관리에 활용
Infra	AWS, Docker	서비스 운영 및 배포 환경 구성에 활용
협업 / 형상관리	GitHub, GitHub Actions	프로젝트 협업과 소스코드 버전 관리, 빌드·테스트·배포 자동화에 활용

3) 전체 시스템 구성도



3.3 주요 엔진 및 기능 설계

3.3.1 프론트엔드

사용자 중심의 단계별 프로젝트 진행 UI 설계

ProMate의 프론트엔드는 프로젝트 경험이 부족한 대학생 팀도 복잡한 사용법 학습 없이 프로젝트를 시작할 수 있도록, 단계별 흐름 중심의 UI 구조로 설계하였습니다.

사용자는 하나의 긴 입력 폼을 작성하는 방식이 아니라, AI 챗봇과 대화하는 형태로 프로젝트 정보를 순차적으로 입력할 수 있으며, 현재까지 정리된 프로젝트 상태를 실시간으로 확인할 수 있습니다.

특히 프로젝트 목표, 역할, 일정, 산출물과 같은 핵심 정보는 세션 요약 영역에 자동 반영되도록 구성하여, 사용자가 현재 프로젝트 진행 상태와 아직 결정되지 않은 항목을 직관적으로 파악할 수 있도록 하였습니다.

이를 통해 단순 채팅 인터페이스가 아니라, 프로젝트 진행 자체를 시각적으로 안내하는 협업 UI를 구현하였습니다.

실시간 협업을 위한 채팅 인터페이스 구현

팀 프로젝트 특성상 여러 명의 사용자가 동시에 의견을 주고받는 환경을 고려하여, 프론트엔드는 SockJS와 STOMP 기반의 실시간 채팅 구조를 적용하였습니다.

사용자는 동일한 프로젝트 채팅방에 참여하여 메시지를 주고받을 수 있으며, AI 응답 또한 동일한 채팅 흐름 안에서 자연스럽게 제공됩니다.

또한 AI가 모든 대화를 무조건 분석하는 것이 아니라, 사용자가 중요하다고 판단한 메시지만 AI 분석 대상으로 전달할 수 있도록 체크박스 기반 UX를 설계하였습니다.

이를 통해 산발적인 대화 속에서도 핵심 맥락만 AI가 반영할 수 있도록 하였으며, 불필요한 정보로 인해 AI 응답 품질이 저하되는 문제를 줄이고자 하였습니다.

상태 관리 및 API 연동 구조 설계

프론트엔드는 Zustand 기반 상태 관리 구조를 사용하여 프로젝트 정보, 사용자 세션, 채팅 상태, 세션 요약 데이터를 관리합니다.

프로젝트 정보와 AI 응답 결과가 여러 화면에서 동시에 참조되기 때문에, 전역 상태 기반 구조를 적용하여 데이터 흐름의 일관성을 유지하였습니다.

프론트엔드와 백엔드, AI 서버 간 통신은 REST API와 WebSocket을 혼합하여 구성하였습니다.

일반적인 프로젝트 생성·조회·수정 기능은 REST API를 사용하고, 채팅과 AI 응답은 실시간 반영이 가능하도록 WebSocket 기반으로 처리하였습니다.

이를 통해 사용자 입력 → 백엔드 전달 → AI 분석 → 결과 반환까지의 흐름이 하나의 자연스러운 인터랙션처럼 보이도록 구현하였습니다.

반응형 UI 및 배포 환경 구성

프론트엔드는 Next.js와 React 기반으로 구현하였으며, Tailwind CSS를 활용해 빠른 UI 개발과 일관된 디자인 시스템을 구성하였습니다.

다양한 환경에서 접근 가능한 웹 기반 서비스를 목표로 하여 반응형 레이아웃을 적용하였습니다.

배포는 Vercel 환경에서 수행하였으며, GitHub 연동 기반 CI/CD를 통해 코드 변경 시 자동 배포가 가능하도록 구성하였습니다.

3.3.2 백엔드

대규모 트래픽과 AI 연동을 고려한 MSA 지향 설계

가장 먼저, 사용자들의 실시간 채팅 트래픽과 상대적으로 응답 지연(Latency)이 긴 AI 연산을 분리하기 위해 마이크로서비스 아키텍처(MSA) 지향적 구조를 채택했습니다. 비즈니스 로직과 세션 관리, 데이터베이스 트랜잭션을 담당하는 메인 API 서버는 안정적인 멀티스레드 처리에 강점이 있는 Java 17과 Spring Boot 3.x 환경으로 구축했습니다. 반면, 복잡한 프롬프트 처리와 LLM 모델 호출은 분리된 FastAPI 서버가 담당합니다. 메인 서버는 WebClient를 통해 AI 서버와 비동기/논블로킹 방식으로 통신하여, AI가 답변을 생성하는 동안에도 메인 서버의 스레드 풀이 고갈되지 않도록 서버 간 결합도를 낮추었습니다.

확장 가능한 실시간 통신 환경 구축

팀원 간의 원활한 소통을 위해 단순한 양방향 통신(WebSocket)을 넘어, 프로젝트 방(Room) 단위로 메시지를 주고받을 수 있는 STOMP 프로토콜을 도입했습니다. 하지만 사용자가 늘어나 서버를 스케일아웃(Scale-out)할 경우, 서로 다른 서버에 접속한 유저 간에 메시지가 전달되지 않는 문제가 발생할 수 있습니다. 이를 해결하기 위해 Amazon ElastiCache(Redis)를 외부 메시지 브로커로 도입했습니다. 사용자가 메시지를 보내면 DB에 안전하게 우선 저장된 후 Redis 채널로 발행(Publish)되고, 각 서버가 이를 구독(Subscribe)하여 연결된 사용자들에게 브로드캐스팅하는 방식으로 무중단 수평 확장이 가능한 채팅 인프라를 완성했습니다.

데이터 무결성 보장과 AI 내결합성 확보

여러 명의 팀원이 동시에 AI 요약(@mates)을 호출할 경우, AI 서버에 과부하가 걸리거나 데이터베이스가 잘못 덮어씌워지는 데이터 경합이 발생할 수 있습니다. 이를 방지하고자 Redis 인프라를 활용해 Redisson 분산 락을 구현했습니다. 특정 프로젝트를 식별하는 Key에 락을 걸어, 한 번의 AI 호출이 처리되는 동안에는 타 사용자의 중복 호출을 차단해 트랜잭션 충돌을 원천 봉쇄했습니다. 또한, AI가 지정된 JSON 스키마를 벗어난 엉뚱한 대답을 내놓더라도 시스템이 멈추지 않도록 방어 로직을 세웠습니다. 파싱 오류나 타임아웃 발생 시 서버를 다운시키는 대신 예외를 잡아내고, 데이터 유실 없이 '가장 마지막에 성공한 요약 상태'를 유지하는 Fallback 전략을 적용하여 시스템의 생존성을 높였습니다.

보안을 강화한 AWS 인프라와 무중단 배포 자동화

이 모든 시스템은 보안과 고가용성을 원칙으로 설계된 AWS 클라우드 위에서 동작합니다. VPC 내에서 Public과 Private Subnet을 엄격히 분리하여, 애플리케이션(EC2)과 데이터베이스(RDS, Redis)는 외부에서 직접 접근할 수 없도록 프라이빗 망에 격리했습니다. 또한, Stateless 기반으로 구성된 EC2 서버와 Amazon S3 기반의 파일 스토리지를 통해 향후 트래픽 증가 시 유연하게 대응할 수 있습니다. 나아가 개발팀의 빠른 이터레이션과 안정적인 서비스 제공을 위해 GitHub Actions와 Docker를 활용한 CI/CD 파이프라인을 구축했습니다. 코드가 병합되면 자동으로 빌드 및 테스트가 수행되고 운영 환경에 무중단으로 배포되도록 자동화하여, 초보 대학생 팀들이 언제나 쾌적하게 프로젝트에 집중할 수 있는 환경을 제공하고 있습니다.

3.3.3 AI

AI 소프트웨어 구조는 API Layer, AI Engine Layer, RAG Layer 의 3계층으로 설계되었으며, 외부 Spring 백엔드와 HTTP로 통신합니다.

최상위 API Layer는 FastAPI 프레임워크 기반으로, Spring 백엔드로부터 HTTP 요청을 수신하고 Pydantic 모델을 통해 입력을 검증합니다. 요청 페이로드는 request_normalization.py를 거쳐 내부 통일 형식으로 정규화된 뒤 AgentState 딕셔너리로 변환되어 하위 계층으로 전달됩니다.

중간의 AI Engine Layer는 LangGraph 라이브러리로 구현된 상태 머신이 action_type과 대화 단계(phase)를 기준으로 적절한 처리 노드로 분기합니다. 각 노드는 Claude LLM을 호출하여 팀 조율자 역할의 응답을 생성하고, 생성된 데이터 업데이트 후보는 apply_collected_data_updates() 함수의 다단계 검증을 거쳐 최종 승인 또는 거부됩니다.

최하위 RAG Layer는 Pinecone 클라우드 벡터 DB와 OpenAI 임베딩 모델을 조합하여 현재 대화 맥락과 관련성이 높은 참고 문서를 실시간으로 검색합니다. 검색된 문서는 LLM 프롬프트에 삽입되어 응답의 도메인 품질을 향상시킵니다.

[AI 엔진 핵심 구성 요소]

구성 요소	위치	역할
-------	----	----

LangGraph Workflow	app/ai/graph/workflow.py	대화 단계별 노드 분기 및 실행 흐름 제어
LLM Decision Core	app/ai/graph/nodes.py	프롬프트 조립, LLM 호출, 응답 파싱
CollectedData Engine	app/ai/graph/collected_data.py	필드 검증·정규화·딴어쓰기 정책·단계 추론
RAG Pipeline	app/rag/retriever.py, vectorstore.py	벡터 유사도 검색, 컨텍스트 포매팅
Request Normalizer	app/core/request_normalization. py	Spring ↔ AI 서버 간 API 필드 호환 변환

LangGraph Workflow는 전체 대화 흐름의 진입점이자 라우터 역할을 수행하며, LLM Decision Core는 실제 언어 모델 호출과 응답 구조화를 담당합니다. CollectedData Engine은 팀의 프로젝트 정보(주제·목표·인원·역할·일정·산출물)가 올바르게 수집·저장되도록 보장하는 데이터 무결성 계층이며, RAG Pipeline은 도메인 지식을 동적으로 주입하는 역할을 합니다. Request Normalizer는 외부 Spring 서버의 요청 형식을 AI 내부 형식으로 흡수하여 두 시스템 간 결합도를 낮춥니다.

[대화 단계 정의]

Phase	단계명	전환 조건	AI 역할
EXPLORE	문제 탐색	초기 상태 (subject 없음)	팀이 다루고 싶은 문제 영역을 자연스럽게 좁혀가도록 유도

TOPIC_SET	주제 확정	title만 존재, subject 미확정	프로젝트 주제를 구체적 서비스/시스템 형태로 정의하도록 유도
PROBLEM_DEFINE	문제 정의	subject는 있으나 추상적 (구체성 부족)	problemArea·targetFacility·targetUser 맥락 수집 후 목표 정의
GATHER	정보 수집	subject 또는 실행 필드 일부 존재	goal·teamSize·roles·dueDate·deliverables 결정 코치
READY	생성 준비	7개 필수 필드 모두 유효값 충족	기획/개발 템플릿 자동 생성

3.4 주요 기능의 구현

3.4.1 프론트엔드

기능 1. AI 챗봇 기반 프로젝트 진행 UI 구현

ProMate의 프론트엔드는 사용자가 복잡한 입력 폼을 한 번에 작성하는 대신, AI 챗봇과의 대화를 통해 프로젝트 정보를 단계적으로 입력할 수 있도록 구현하였습니다.

사용자가 프로젝트 주제 존재 여부를 선택하면 해당 상태에 따라 다른 질문 흐름이 화면에 출력되며, 이후 목표, 역할, 일정, 산출물 등 프로젝트 진행에 필요한 항목이 순차적으로 수집됩니다.

또한 AI 응답과 사용자 입력은 하나의 채팅 흐름 안에서 자연스럽게 연결되도록 구성하였으며, 사용자는 현재 어떤 항목이 정리되었고 어떤 항목이 아직 미정 상태인지 실시간으로 확인할 수 있도록 구현하였습니다.

기능 2. 세션 요약(Session Summary) 및 프로젝트 상태 관리 기능 구현

프론트엔드는 프로젝트 진행 과정에서 수집된 정보를 세션 요약 영역에 반영할 수 있도록 구현하였습니다. 세션 요약에는 프로젝트 제목, 목표, 팀원 역할, 일정, 산출물과 같은 핵심 항목이 포함되며, 사용자는 이를 통해 현재까지 정리된 프로젝트 상태를 한눈에 확인할 수 있습니다.

세션 요약은 AI 챗봇과 팀원들의 대화 내용을 기반으로 자동으로 채워질 수 있으며, 필요한 경우 사용자가 직접 내용을 입력하거나 수정하는 방식으로 관리할 수 있습니다. 이를 통해 AI가 정리한 결과를 그대로 사용하는 것뿐만 아니라, 팀의 실제 의사결정에 맞게 사용자가 직접 보완할 수 있도록 하였습니다.

또한 아직 결정되지 않은 항목은 미정 상태로 남겨두어, 팀이 다음으로 무엇을 논의해야 하는지 쉽게 파악할 수 있도록 구성하였습니다. 이를 통해 단순한 채팅 화면이 아니라, 프로젝트 진행 상태를 구조적으로 확인하고 관리할 수 있는 협업 UI를 구현하였습니다.

기능 3. 실시간 팀 채팅 및 AI 연동 인터페이스 구현

프론트엔드는 SockJS와 STOMP 기반 WebSocket 구조를 활용하여 실시간 채팅 기능을 구현하였습니다.

동일한 프로젝트 방(Room)에 참여한 팀원들은 동시에 메시지를 주고받을 수 있으며, AI 응답 또한 동일한 채팅 흐름 안에서 실시간으로 출력됩니다.

또한 여러 사용자가 동시에 대화하는 상황에서 AI가 핵심 메시지만 분석할 수 있도록 체크박스 기반 인터페이스를 구현하였습니다. 사용자가 중요하다고 판단한 메시지에 체크하면, 해당 메시지만 AI 분석

대상으로 전달되도록 구성하여 불필요한 대화로 인해 AI 응답 품질이 저하되는 문제를 줄이고자 하였습니다.

기능 4. 사용자 인증 및 프로젝트 관리 화면 구현

Google OAuth2 로그인 기반 사용자 인증 화면을 구현하였으며, 로그인 이후 사용자는 자신의 프로젝트 목록 조회, 프로젝트 생성, 초대 코드 기반 프로젝트 참여 기능을 사용할 수 있도록 구성하였습니다.

또한 프로젝트별 채팅방 진입, 세션 요약 조회, 템플릿 생성 요청과 같은 주요 기능을 하나의 흐름 안에서 수행할 수 있도록 화면 구조를 설계하였습니다.

기능 5. API 연동 및 상태 관리 구현

프론트엔드는 REST API와 WebSocket을 함께 사용하여 백엔드 및 AI 서버와 통신하도록 구현하였습니다.

프로젝트 생성·조회·수정과 같은 일반 기능은 REST API 기반으로 처리하고, 채팅 및 AI 응답은 WebSocket 기반 실시간 통신으로 처리하였습니다.

또한 Zustand 기반 전역 상태 관리 구조를 적용하여 사용자 정보, 프로젝트 상태, 채팅 데이터, 세션 요약 데이터를 관리하였다. 이를 통해 여러 화면에서 동일한 프로젝트 데이터를 일관성 있게 사용할 수 있도록 구현하였습니다.

3.4.2 백엔드

구성 요소 (도메인)	위치	역할
Project	POST /api/projects	새 프로젝트 생성

(프로젝트 관리)		
	POST /api/projects/join	초대 코드를 통한 프로젝트 참여
	GET /api/projects/me	로그인한 유저의 내 프로젝트 목록 조회
	GET /api/projects/{projectId}	프로젝트 상세 조회 (정형화된 세션 요약 데이터 포함)
	DELETE /api/projects/{projectId}	프로젝트 삭제 (방장 전용 권한)
	GET /api/projects/{projectId}/members	해당 프로젝트의 팀원 목록 및 초대 코드(inviteCode) 조회
	GET /api/projects/{projectId}/summary	프로젝트의 세션 요약 정보만 단독 조회
	PATCH /api/projects/{projectId}/summary	프로젝트 정보 및 세션 요약 수동 업데이트
	POST /api/projects/{projectId}/export	기획 완료 후 AI 분석 실행 및 Notion으로 결과물 내보내기
Chat (실시간 채팅)	GET /api/chat/{projectId}/messages	특정 프로젝트 방의 과거 채팅 내역(History) 조회

	POST /api/chat/{projectId}/messages	메시지 전송 폴백 (웹소켓 연결 불가 시 REST API를 통한 전송)
User (회원 관리)	GET /api/users/me	현재 로그인된 유저의 정보 조회 (SYS-01)
Manual (매뉴얼 관리)	GET /api/manuals	서비스 사용 매뉴얼 목록 조회
	POST /api/manuals	서비스 사용 매뉴얼 등록
AI (AI 연동)	POST /ai/project/templates	AI 모델을 활용한 프로젝트 템플릿 생성 요청
System (서버 상태)	GET /health	AWS 로드밸런서(ALB) 및 CI/CD를 위한 서버 Health Check

안전한 데이터 통신을 위해 엔티티(Entity)를 직접 노출하지 않고, 다음과 같은 DTO(Data Transfer Object) 스키마를 구성하여 응답과 요청을 분리했습니다.

- Request (요청): CreateRequest, ExportRequestDto, JoinRequest, SummaryUpdateRequest

- Response (응답): CreateResponse, ManualResponseDto, ChatMessageDto, DetailResponse, SessionSummaryDto, InfoResponse, TeamResponse, ListResponse

3.4.3 AI

기능 1. LangGraph 기반 대화 흐름 관리 및 LLM 의사결정

ProMate는 사용자의 프로젝트 준비 상태에 따라 적절한 대화 흐름을 제공하기 위해 LangGraph 기반 상태 흐름을 구현하였습니다. 사용자가 “주제가 없다”를 선택하면 아이디어 탐색 흐름으로, “주제가 있다”를 선택하면 주제 구체화 흐름으로 이동합니다. 또한 기획 템플릿이나 개발 템플릿 생성을 요청하면 해당 생성 노드가 실행되며, 별도의 버튼 입력이 없는 경우에는 현재까지 수집된 프로젝트 정보의 완성도를 기준으로 다음 단계를 자동 판단합니다.

각 대화 노드에서는 사용자 메시지, 최근 대화 기록, 기존에 수집된 프로젝트 정보, 아직 비어 있는 항목, 참고 문서 검색 결과를 조합하여 LLM 프롬프트를 구성합니다. LLM은 사용자에게 보여줄 답변뿐 아니라 대화 의도, 응답 방식, 새롭게 저장할 수 있는 프로젝트 정보 후보, 다음으로 수집해야 할 항목을 JSON 구조로 반환합니다. 이를 통해 자연어 응답과 시스템 내부 데이터가 안정적으로 연결되도록 하였습니다.

LLM 응답은 그대로 사용하지 않고 파싱 및 검증 과정을 거칩니다. 응답 형식이 올바르지 않거나 필수 필드가 누락된 경우에는 기본 오류 메시지를 반환하여 비정상적인 출력이 발생하지 않도록 하였습니다.

기능2. 프로젝트 정보 수집 및 검증

ProMate는 대화 중 사용자가 입력한 프로젝트 정보를 자동으로 추출하고 저장합니다. 수집 대상 정보에는 프로젝트 주제, 목표, 팀 인원, 역할, 마감일, 산출물 등이 포함됩니다. 다만 LLM이 생성한 값은 바로 저장하지 않고, 별도의 검증 과정을 거친 뒤 승인된 값만 최종 데이터에 반영합니다.

예를 들어 “팀원은 4명입니다”라는 입력은 숫자 값으로 정규화하고, “프론트엔드, 백엔드, 기획”과 같은 입력은 역할 목록으로 분리하여 저장합니다. 반대로 “아직 미정입니다”, “나중에 정하겠습니다”와 같은 표현은 유효한 프로젝트 정보로 보기 어렵기 때문에 저장하지 않습니다.

기능 3. RAG 기반 참고자료 활용

ProMate는 AI 응답의 품질을 높이기 위해 현재 대화 맥락과 관련된 참고 문서를 검색하여 LLM 프롬프트에 함께 제공합니다. 사용자가 프로젝트 아이디어를 탐색하는 단계에서는 문제 정의나 가치 제안 관련 자료를 활용하고, 개발 템플릿을 생성하는 단계에서는 API 설계나 소프트웨어 공학 기준과 관련된 자료를 우선적으로 활용합니다.

검색된 문서는 그대로 사용하지 않고, 출처와 핵심 내용이 포함된 짧은 텍스트 형태로 정리한 뒤 프롬프트에 삽입합니다. 또한 LLM 입력 길이가 지나치게 길어지는 것을 방지하기 위해 문서별 길이와 전체 참고자료 길이에 제한을 두었습니다.

이 기능을 통해 ProMate는 일반적인 답변이 아니라, 프로젝트 기획과 개발에 필요한 방법론과 기준을 반영한 응답을 제공하도록 구현했습니다.

기능 4. 프로젝트 템플릿 자동 생성

대화를 통해 프로젝트 정보가 충분히 수집되면, ProMate는 사용자의 요청에 따라 기획 템플릿 또는 개발 템플릿을 자동으로 생성합니다. 템플릿 생성 시에는 현재까지 승인된 프로젝트 정보만 사용하며, 불완전하거나 검증되지 않은 값은 제외합니다.

기획 템플릿은 프로젝트 개요, 문제 정의, 목표, 주요 기능, 기대 효과 등을 중심으로 구성됩니다. 개발 템플릿은 시스템 구조, 주요 기능 구현 방식, 기술 스택, API 또는 모듈 설계 내용 등을 포함하도록 생성됩니다. 생성된 결과는 Notion 연동에 사용할 수 있는 JSON 형식으로 변환되어 Spring 백엔드로 전달됩니다.

또한 LLM 호출에 실패하더라도 기본 템플릿 구조를 반환하도록 하여, 외부 API 오류가 발생하더라도 사용자가 최소한의 결과물을 받을 수 있도록 처리하였습니다.

3.5 기타 구현 사항

아래는 AI 주요 기능 외 안정성, 호환성 사용성 향상을 위해 구현한 보조 기능입니다.

- **Spring 요청 형식 정규화**

Spring 백엔드와 AI 서버 사이의 필드 이름 및 요청 구조 차이를 줄이기 위해 요청 정규화 처리를 구현하였습니다. 예를 들어 Spring에서 전달되는 프로젝트 ID, 버튼 타입, 선택 답변, 현재 상태 값을 AI 서버 내부에서 사용하는 형식으로 변환합니다.

이를 통해 Spring 서버의 요청 구조가 일부 변경되더라도 AI 서버의 핵심 로직은 동일한 데이터 형식을 기준으로 동작할 수 있습니다. 또한 일부 필드가 누락된 요청이 들어오더라도 기본값을 채워 처리할 수 있도록 하여 API 연동 안정성을 높였습니다.

- **응답 방식 제어**

대화 상황에 따라 AI 응답 방식이 달라지도록 응답 제어 정책을 적용하였습니다. 예를 들어 정보 수집이 필요한 경우에는 답변 후 다음 질문을 이어가고, 사용자가 이미 충분한 정보를 제공한 경우에는 추가 질문 없이 요약 응답만 제공하도록 하였습니다.

특히 LLM이 불필요하게 마지막에 질문을 덧붙이는 경우를 방지하기 위해, 질문 문장 제거 후처리를 적용하였습니다. 이를 통해 사용자가 요청한 상황에 맞는 응답 형식을 일관되게 유지할 수 있습니다.

3.6 계획

1) 현재까지 수행내용

(1) 기획 및 설계

- 문제 정의 및 타겟 사용자 설정 완료
- 서비스 핵심 기능 3종 정의 완료
- 화면 흐름 및 UI/UX 와이어프레임 설계 완료

(2) 현재 진행 상황

- ~~프론트엔드 UI/UX 구현 진행~~
- ~~백엔드 API 설계 완료~~
- ~~AI 파이프라인 및 RAG 구축 완료~~
- ~~AWS 기반 인프라 및 배포 완료~~
- ~~프론트엔드-백엔드-AI 연동 완료~~

(3) 현재 확인된 과제

- ~~기능 간 API 연동 안정화 필요~~
- ~~AI 응답 속도 및 workflow 흐름 개선 필요~~
- ~~문서/템플릿 생성 결과 품질 보완 필요~~
- ~~실제 사용자 시나리오 기반 검증 강화 필요~~

2) 향후 계획

순번	계획 항목	상세 내용	예정 시기
1	서비스 정의 문구 구체화	서비스 한줄 정의 및 소개 문구 최종 확정	3월 4주
2	응답 속도 개선	RAG 선택적 적용 기준 강화 및 경량 모델 전환 후 성능 비교 실험	3월 4주 ~ 4월 1주
3	파인튜닝 방안 검토	파인튜닝 적용 가능성 분석 및 실험 설계	4월 1주
4	제목 생성 방식 개선	규칙 기반 생성 또는 사용자 직접 입력 방식 비교 검토 및 적용	4월 1주
5	데이터셋 검증 및 보완	Human-in-the-loop 및 AI 교차 검증을 통한 데이터셋 품질 개선	4월 2주

6	1차 실험 및 SI 보완	검증된 데이터셋 기반 1차 실험 진행 후 SI 응답 품질 보완	4월 2주
7	1차 실험 결과 보완	배포 결과 분석 및 SI 응답 품질·workflow 오류 보완	4월 3주
8	2차 서비스 배포 및 실사용자 테스트	보완된 서비스 2차 배포 후 실제 사용자 대상 테스트 진행 및 피드백 수집	4월 3주
9	백엔드·프론트엔드 API 연동 오류 해결	백엔드와 프론트엔드 API 연동시 발생하는 오류 수정	상사